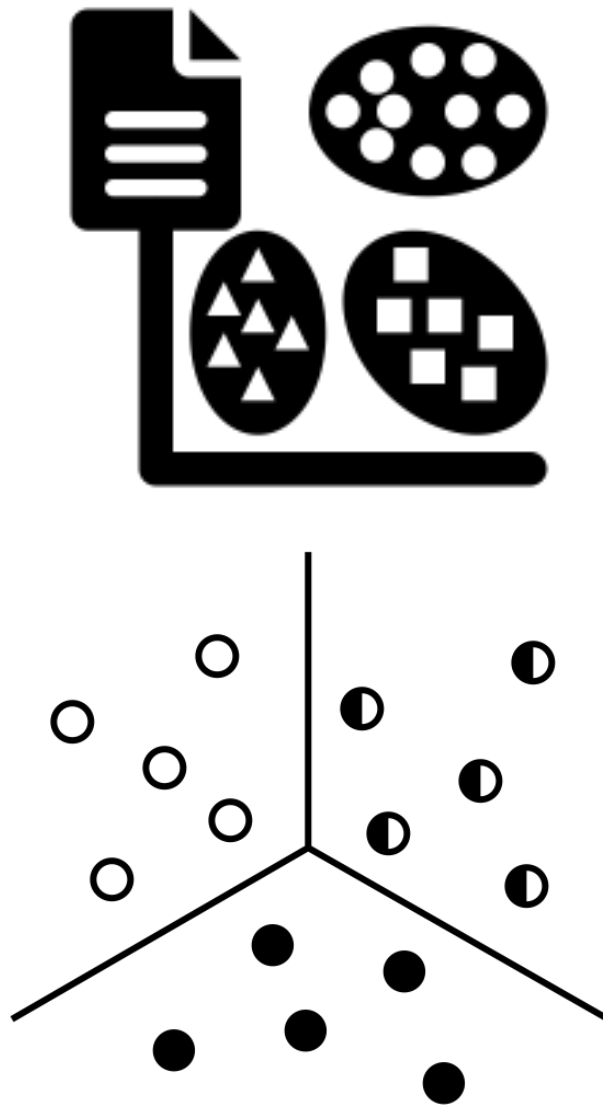




# K-MEANS CLUSTER

INTRO TO AI

Youssef Eslam Hussein | 231000070 | 9/10/2026

# INTRODUCTION TO K-MEANS

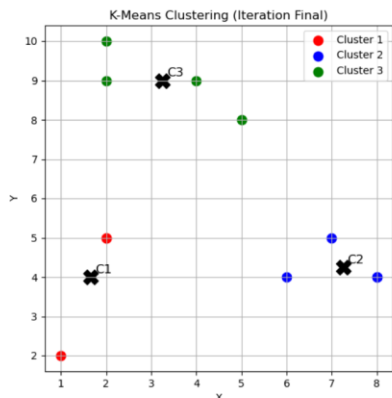
**K-Means clustering algorithm** is an unsupervised machine learning technique that takes Data points and organizes them into groups. Used to partition an unlabeled dataset into **K distinct, non-overlapping groups (clusters)** based on feature similarity. But our cluster isn't just a normal cluster it's a **satellite image cluster** that uses a group pixel with similar spectral values into  $k$  distinct land-cover classes (such as water, vegetation, urban areas, and bare soil) without requiring pre-labeled training data.

## HOW THE ALGORITHM WORKS

Every cluster algorithm differs from one another for example we have in the in the normal data points cluster which we randomly put starting **centroid** points. The goal is to calculate the distance between centroids and other point to know which data point is belonged to which cluster. Then we calculate center of each cluster to point the centroid in the middle of the clusters.

So again, let's see k-means algorithms step-by-step

1. Initialization: We select  $K$ ; initial centroids.
2. Assignment: We calculate the distance between every single pixel in the image and the  $K$  centroids.
3. Centroid Update: Once all pixels are assigned, we recompute the position of each centroid.
4. Convergence Check: We compare the new centroids to the old ones. If they haven't moved (or changed minimally).



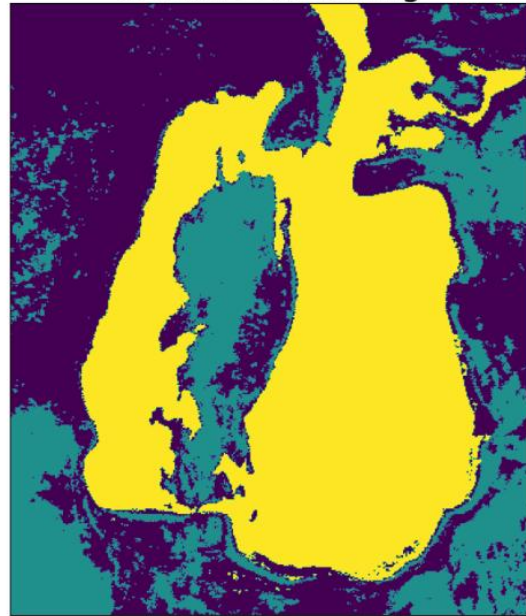
## The Core Concept: Pixels as Data Points

K-means clustering groups satellite image pixels into K distinct clusters based on color or spectral values. Each pixel becomes a data point in a multi-band space. The algorithm assigns pixels to the nearest cluster center, then updates center iteratively until pixel groupings stop changing.

MOD09GA RGB



kMeans Clustering



## Impact of the 'K' Value

The hyperparameter K dictates the number of segments the image will be divided into:

- $K = 3$ : Forces the algorithm to broadly classify the image into the three most dominant features (e.g., deep water, shallow water/vegetation, and desert/land). This provides a macro-level overview of the geography.
- $K = 5$ : Allows for more granular segmentation. The algorithm can distinguish between subtle variations, such as different types of soil, varying depths/sediment levels in water, and dense vs. sparse vegetation.

## Distance Metrics: Euclidean vs. Manhattan

We implemented two specific mathematical approaches to calculate the ‘closeness’ of a pixel to a centroid using **Euclidean Distance** & **Manhattan Distance**.

### EUCLIDEAN DISTANCE (L<sub>2</sub> NORM)

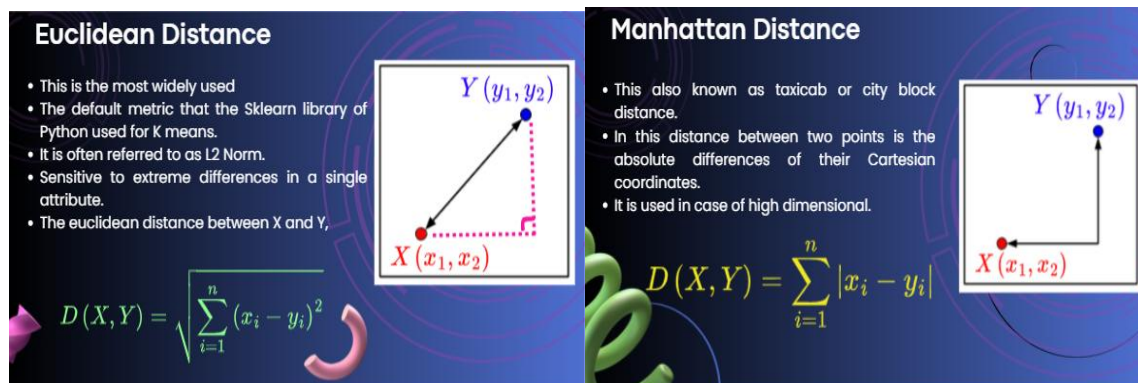
Euclidean distance tends to form spherical clusters and heavily penalizes large differences in any single-color channel due to the squaring operation.

- **Data Preparation:** Reshape the multi-band satellite raster (e.g., RGB or multispectral bands like NIR, Red, Green) into a 2D array where each row is a pixel and columns are spectral features.
- **Initialization:** Choose K and pick random initial cluster centers (centroids) in the multi-dimensional spectral space.
- **Distance Calculation:** Compute the Euclidean distance (L<sub>2</sub> norm)  $d(p, c) = \sqrt{\sum (p_i - c_i)^2}$  between each pixel vector  $p$  and centroid  $c$ .
- **Assignment & Update:** Assign each pixel to the nearest centroid, then recompute centroids as the mean of all assigned pixels. Repeat until convergence.

### MANHATTAN DISTANCE (L<sub>1</sub> NORM)

Manhattan distance is generally more robust to outliers than Euclidean distance and tends to form diamond-shaped clusters in the feature space. **How K-Means Works with Manhattan Distance**

- **Pixel Representation:** Each pixel is a vector in a multi-dimensional space (e.g., Red, Green, Blue, or multi-spectral bands like NIR).
- **Distance Formula:** Computes the sum of absolute differences between pixel values and cluster centroids:  $D = \sum |x_i - y_i|$
- **Centroid Update:** Uses the median instead of the mean during the update step to strictly minimize the Manhattan objective, though standard K-Means variants sometimes retain the mean



# CODING

This is the coding structure

| Layer / Component              | Function / Class                     | Primary Responsibility                                                        | Key Algorithms & Techniques                                                                                                                                                                                                                                                                                                                                |
|--------------------------------|--------------------------------------|-------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Core AI Engine                 | <code>kmeans()</code>                | Implements the unsupervised K-Means algorithm from scratch.                   | <ul style="list-style-type: none"><li>Centroid initialization (<code>data[:K]</code>)</li><li>Vectorized distance (<code>np.newaxis</code>)</li><li>Euclidean (<code>\$L_2\$</code>) &amp; Manhattan (<code>\$L_1\$</code>) metrics</li><li>Centroid update via mean (<code>np.mean</code>)</li><li>Convergence check (<code>np.allclose</code>)</li></ul> |
| Data Processing & Segmentation | <code>generate_slide_figure()</code> | Preprocesses satellite images and runs comparative segmentation.              | <ul style="list-style-type: none"><li>OpenCV BGR-to-RGB conversion</li><li>Aspect-ratio preserving resize</li><li>Flattening image matrix (<code>(H,W,3)</code> to <code>(N,3)</code>)</li><li>Multi-subplot figure creation with Matplotlib</li></ul>                                                                                                     |
| Render Bridge                  | <code>fig_to_photoimage()</code>     | Converts Matplotlib figures into PIL images in memory without pop-up windows. | <ul style="list-style-type: none"><li>Non-interactive <code>Agg</code> backend rendering</li><li>Memory RGBA buffer extraction</li><li>High-quality downscaling (<code>PIL.Image.LANCZOS</code>)</li></ul>                                                                                                                                                 |
| User Interface (GUI)           | <code>SlideshowApp</code>            | Provides an interactive slideshow window with dark theme styling.             | <ul style="list-style-type: none"><li>Object-Oriented Tkinter layout</li><li>Responsive image scaling</li><li>Keyboard bindings (<code>&lt;Left&gt;</code>, <code>&lt;Right&gt;</code>, <code>&lt;Escape&gt;</code>)</li></ul>                                                                                                                             |
| Concurrency Management         | <code>threading.Thread</code>        | Keeps GUI responsive while heavy image processing runs.                       | <ul style="list-style-type: none"><li>Asynchronous background daemon thread</li><li>Thread-safe UI updates using <code>root.after()</code></li></ul>                                                                                                                                                                                                       |

# Conclusion

By writing this algorithm from scratch using pure NumPy matrix operations, we successfully applied theoretical machine learning concepts to real-world remote sensing data. The vectorized implementation ensures that despite calculating distances for hundreds of thousands of pixels iteratively, the algorithm executes highly efficiently.